

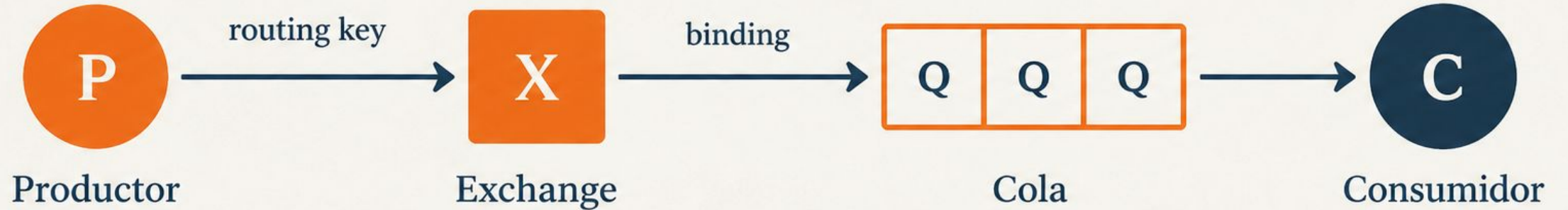
RabbitMQ

Routing

Mensajes según su destino

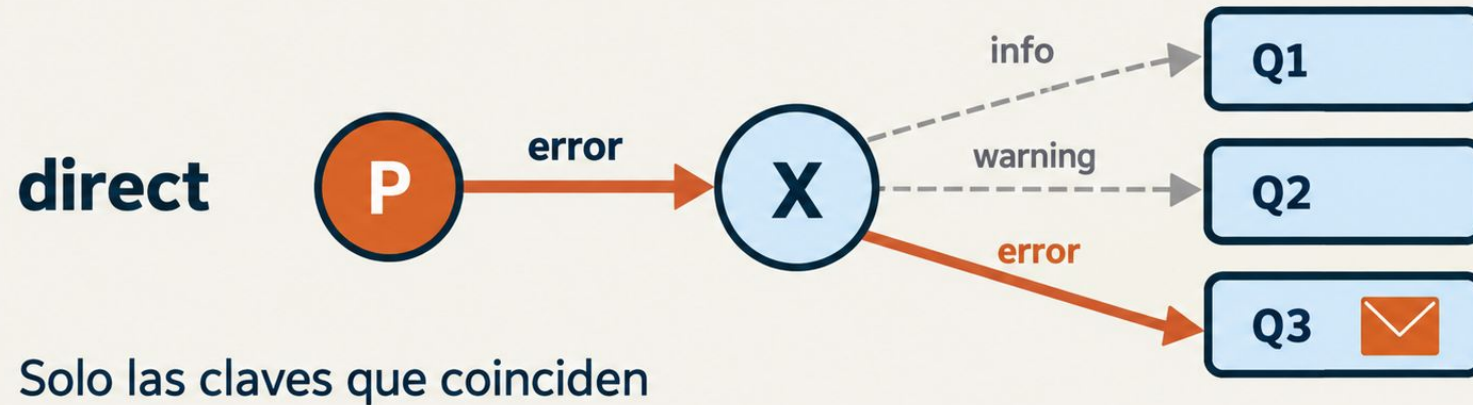
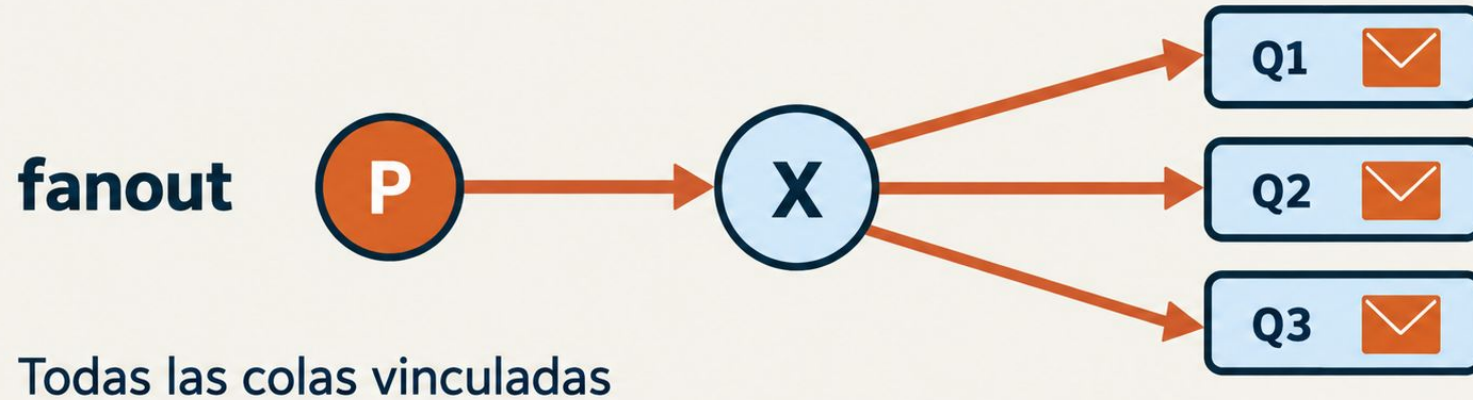
Tutorial 4 · JavaScript · Sistemas Distribuidos

El recorrido de un mensaje



El productor publica en el exchange

Fanout y direct



La regla: coincidencia exacta

routing key = **binding key**

error = **error**

Sucre $\neq$ **sucrer**

Direct no interpreta * ni # como comodines

Varios bindings, varias entregas

Clave publicada	Archivo	Consola
info	—	Sí
warning	Sí	Sí
error	Sí	Sí

Un error → dos colas → dos entregas

Producer: publicar con una clave

```
await channel.assertExchange(  
    'direct_logs', 'direct', { durable: false }  
);
```

```
channel.publish('direct_logs', severity,  
    Buffer.from(message));
```

severity = error

emit_log_direct.js + lib/routing.js

Consumidor: declarar sus intereses

```
const { queue } = await channel.assertQueue(
  '', { exclusive: true });

for (const key of keys)
  await channel.bindQueue(queue, 'direct_logs', key);

await channel.consume(queue, onMessage, { noAck: true });
```

Una cola propia por receptor

La demo del tutorial

1 · Archivo

```
node receive_logs_direct.js warning error
```

2 · Consola

```
node receive_logs_direct.js info warning error
```

3 · Productor

```
node emit_log_direct.js error "Pedido fallido"
```

Primero los consumidores; después el productor

Qué garantiza esta práctica

Mecanismo	Efecto en esta demo
exclusive: true	Cola temporal por conexión
noAck: true	Sin acuse manual del consumidor
Publisher confirms	El broker confirma la publicación
mandatory: true	Devuelve mensajes sin ruta

La confirmación del broker no prueba procesamiento

Cuándo usar direct

Destinos conocidos

Coincidencia exacta

Una o varias colas por clave

¿Qué cambia si otra cola se vincula con sucre?

Producción: persistencia, acuses y manejo de fallos